Data Structures and Algorithms

Lecture 6

Trees

Dept. of ICT, Faculty of Technology- UoR, 2026

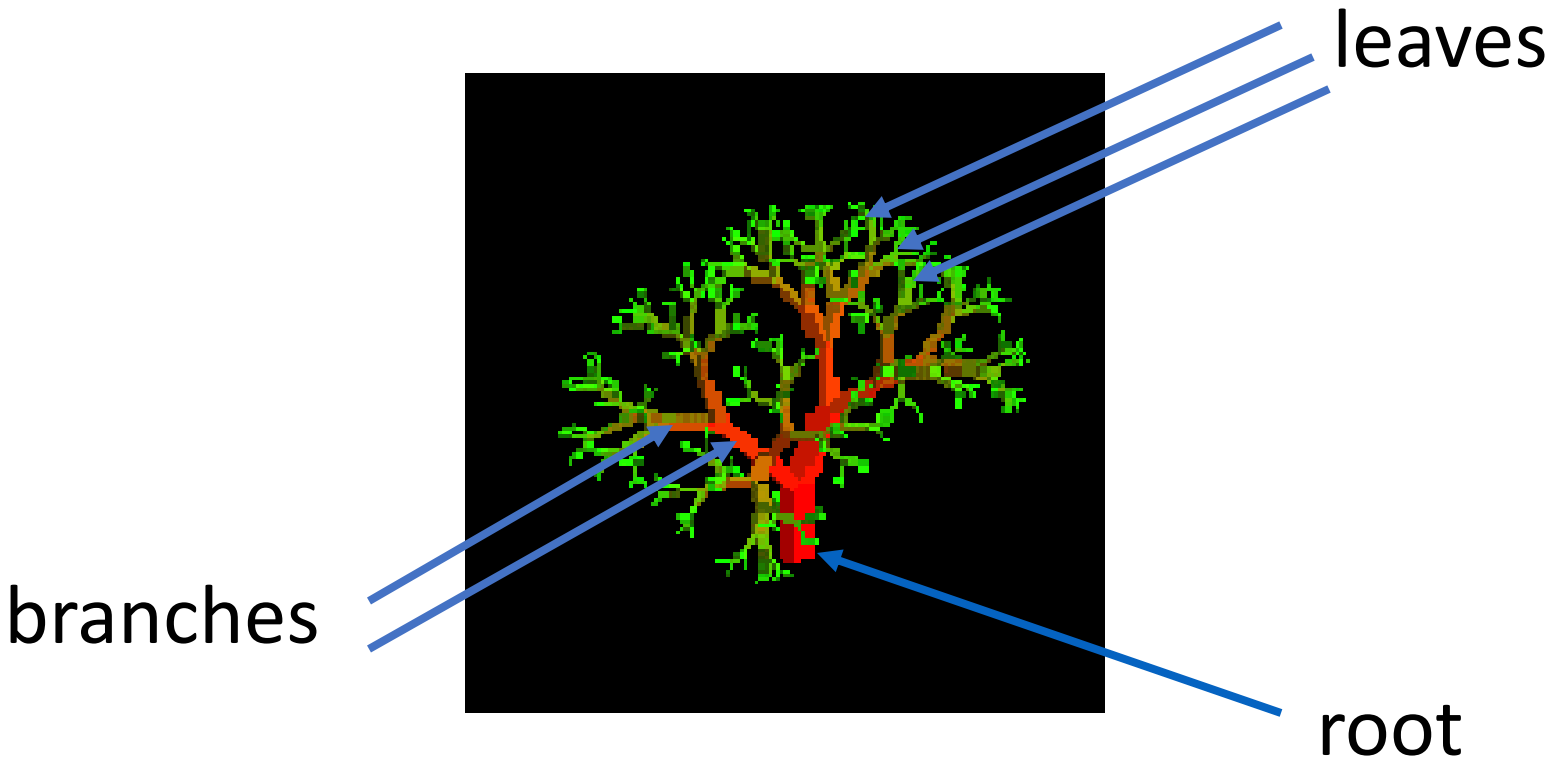
Introduction

- Arrays and Linked Lists are linear structures the time required to search is little high.
- This not acceptable in today's world where speed at which we complete operations is extremely important.
- Therefore it seems that better (more efficient) data structures are needed to store and search data.
- We can extend the concept of linked data structure (linked list, stack, queue) to a structure that may have multiple relations among its nodes. Such a structure is called a **tree**.

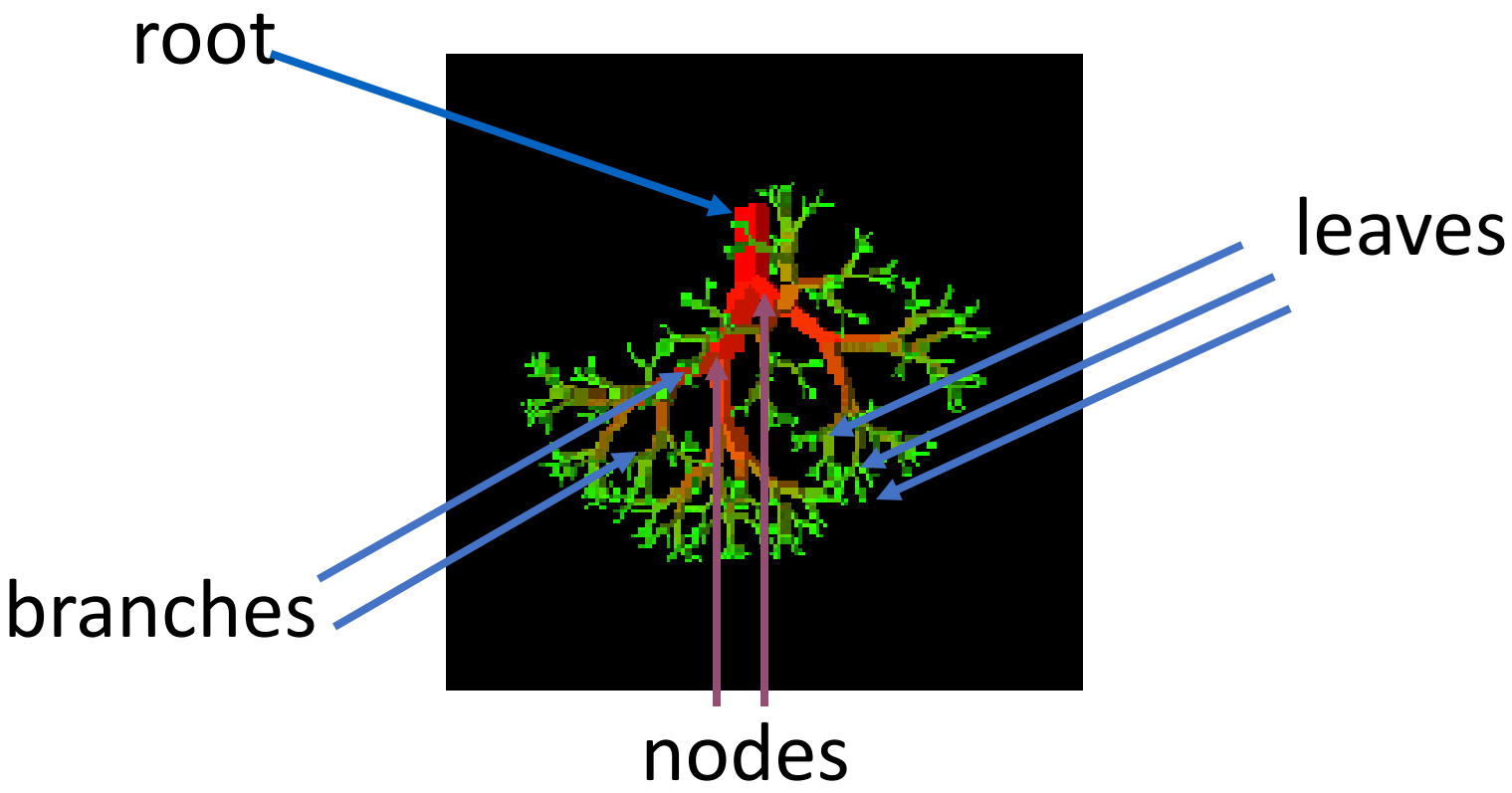
Trees

- A widely-used data structure.
- Not a linear structure (compared to arrays, linked lists, stacks and queues which are linear data structures).
- A tree consists of set of nodes and set of edges that connect pair of nodes. A tree can be empty with no nodes
- Useful for representing hierarchical relationships among data items

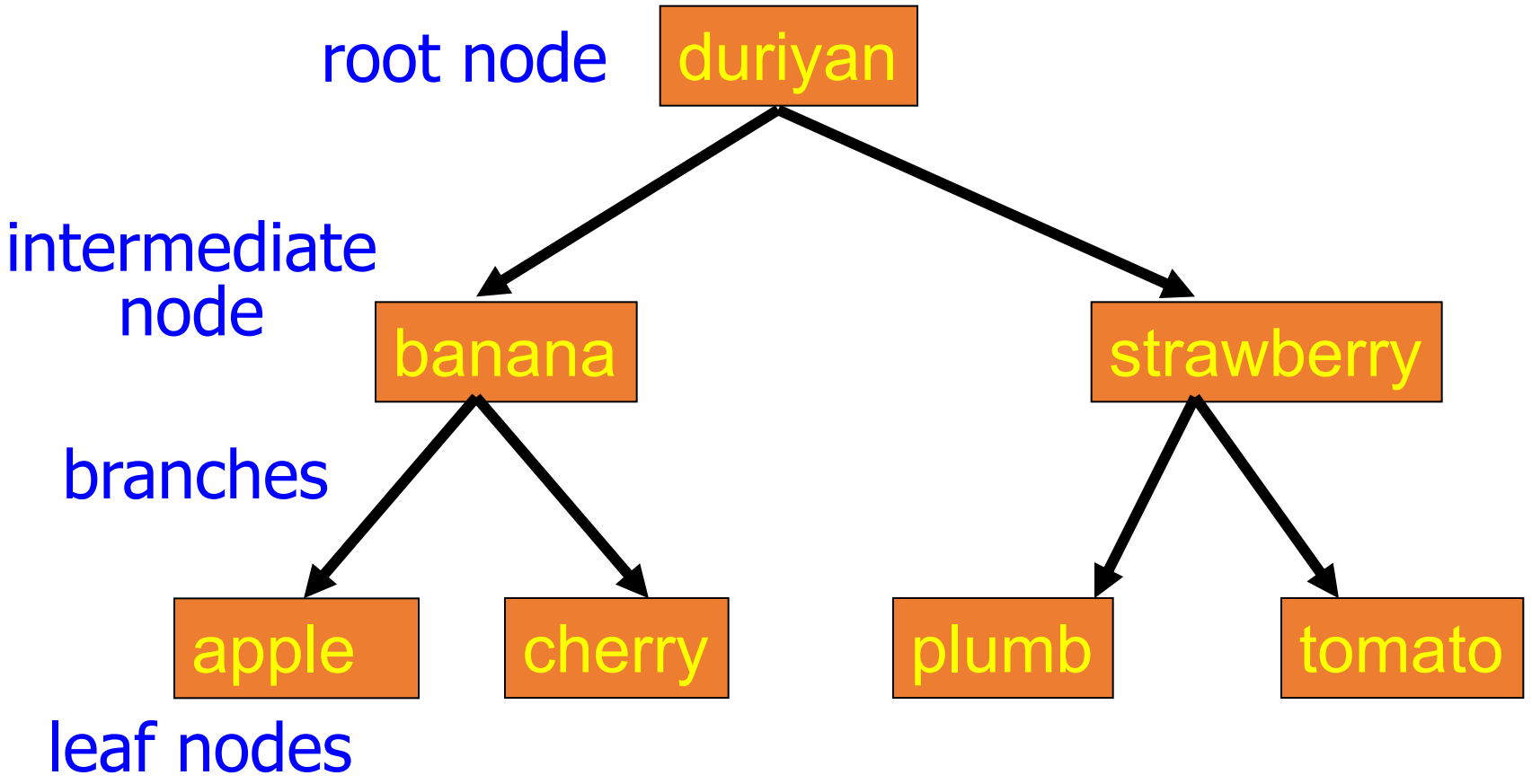
Nature View of a Tree



Computer Scientist's View

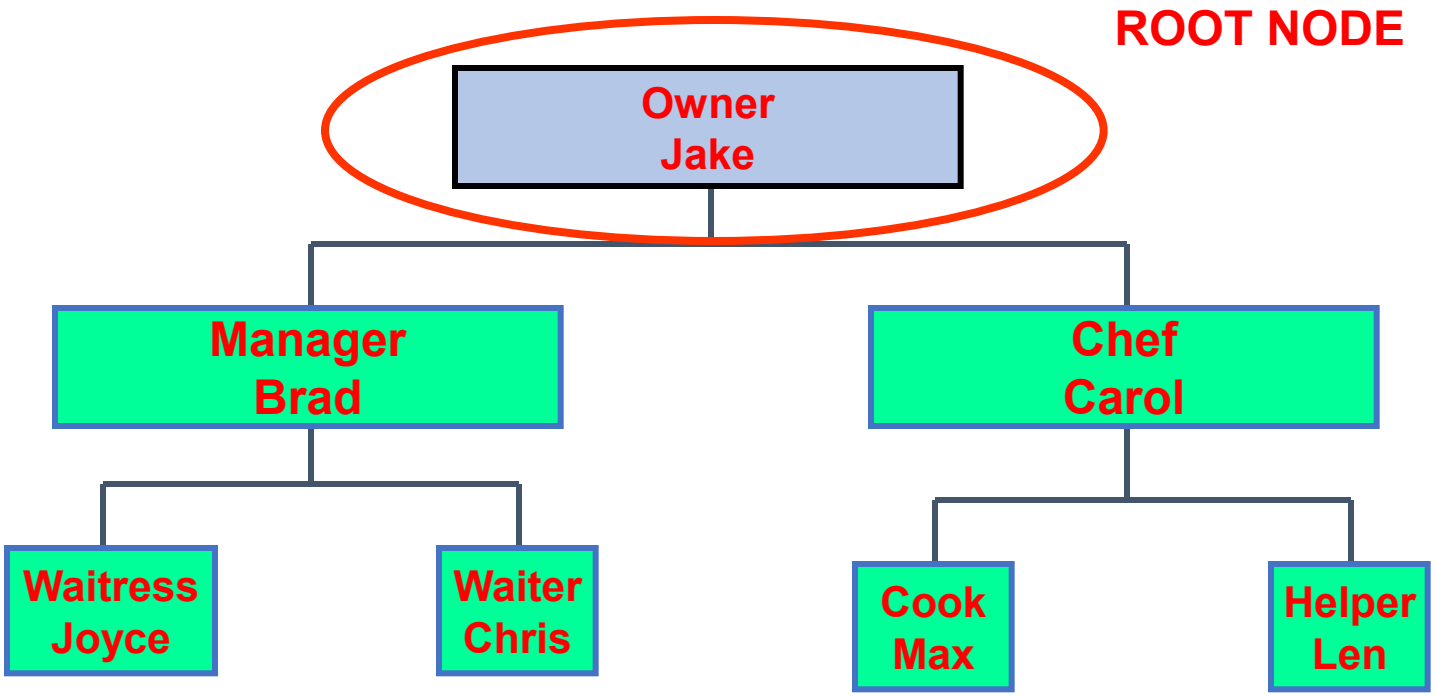


Trees in computing...

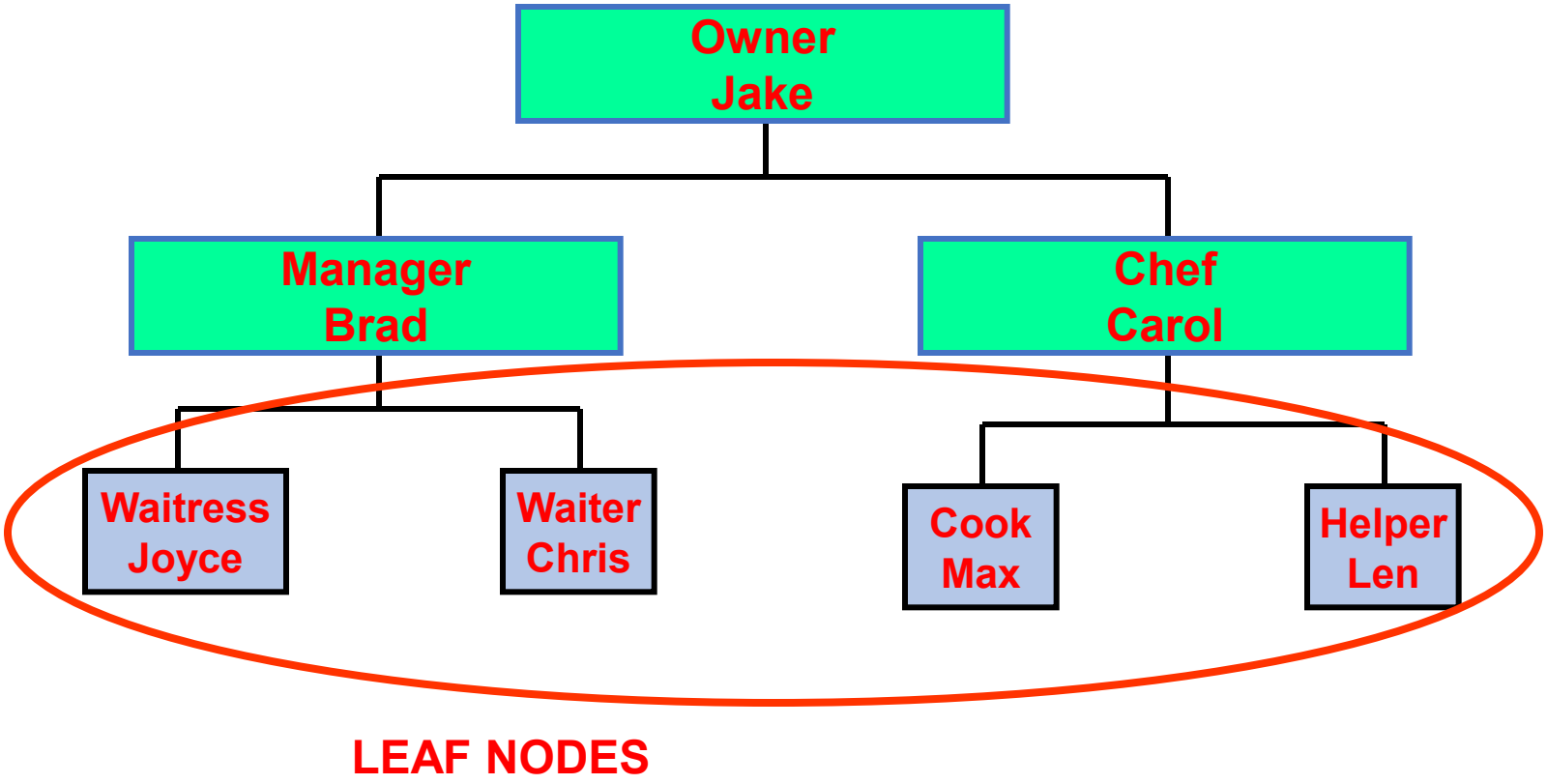


One root at top with many leaves at the bottom.

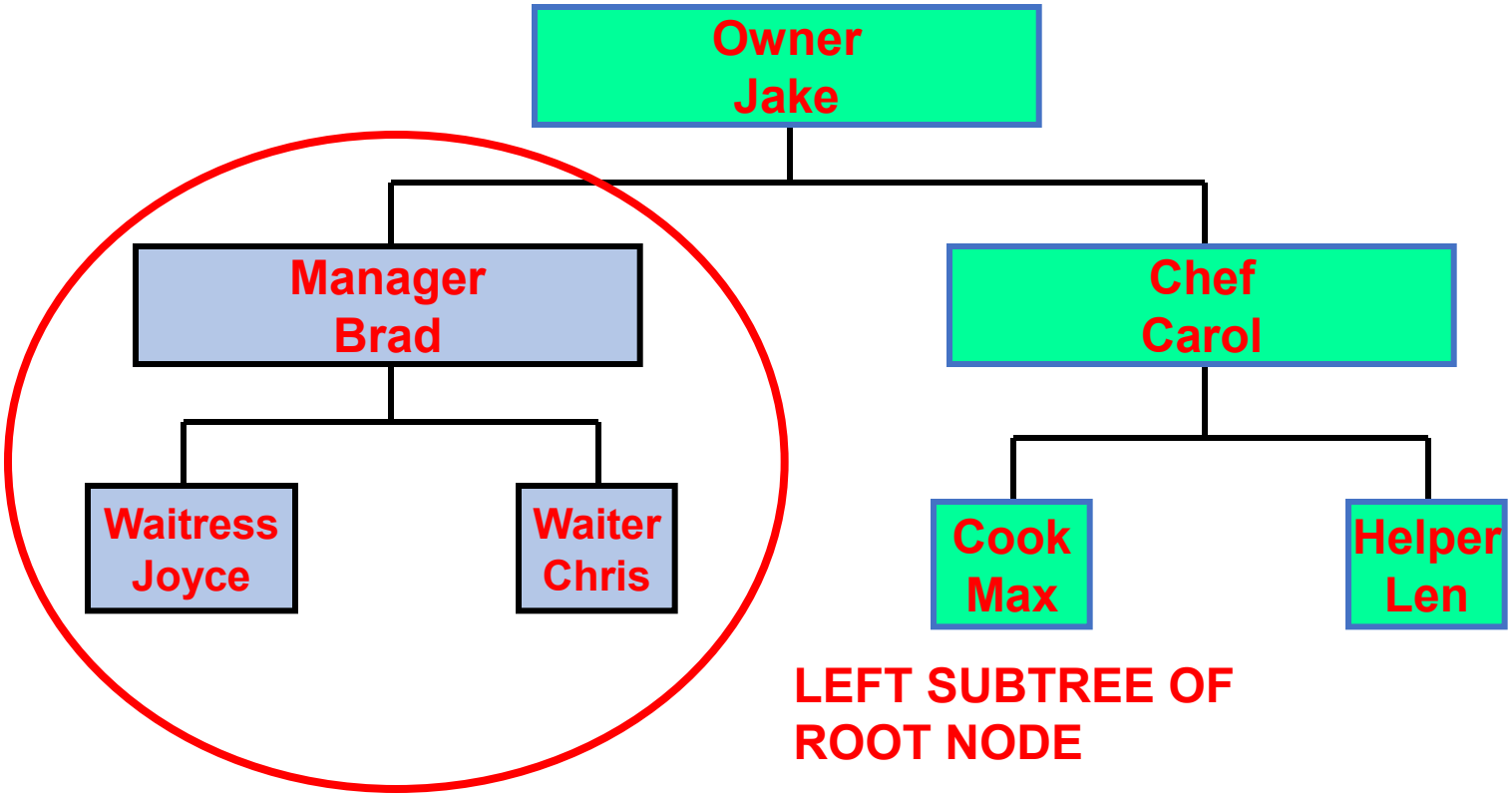
A Tree Has One Root Node



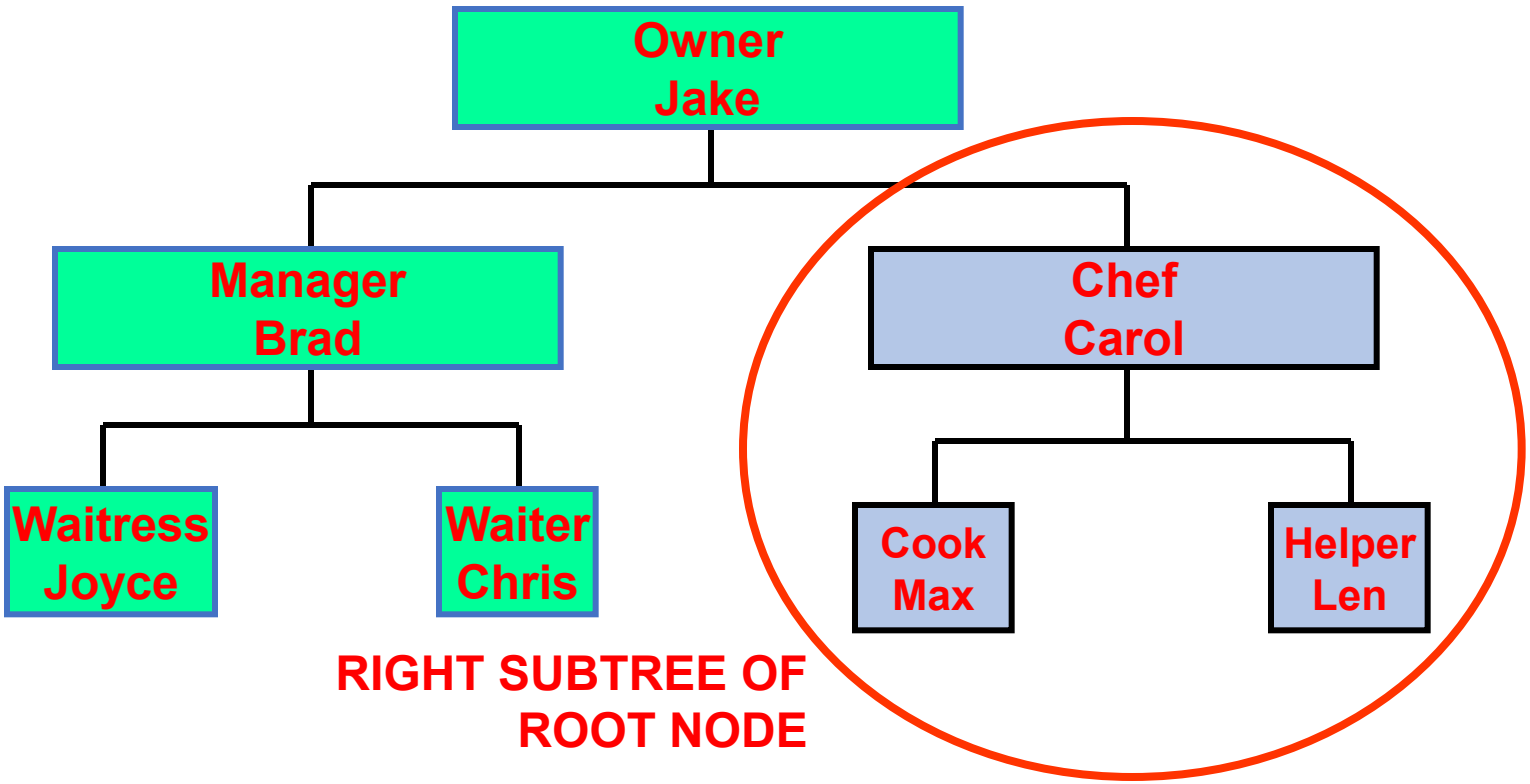
Leaf Nodes Have No Children



A Subtree


Owner
Jake
Manager
Brad
Chef
Carol
Waitress
Joyce
Waiter
Chris
Cook
Max
Helper
Len
LEFT SUBTREE OF
ROOT NODE

A Subtree


Owner
Jake
Manager
Brad
Chef
Carol
Waitress
Joyce
Waiter
Chris
Cook
Max
Helper
Len
RIGHT SUBTREE OF
ROOT NODE

Terminology

- Root
- Parent
- Child
- Sibling
- External node
- Internal node
- Subtree
- Ancestor
- Descendant

Basic Tree Concepts

- Node – user-defined data structure that contains pointers to data and pointers to other nodes:
  - **Root** – Node from which all other nodes descend
  - **Parent** – has child nodes arranged in subtrees.
  - **Child** – nodes in a tree have 0 or more children.
  - **Leaf** – node without descendants
  - **Degree** – number of direct children a tree/subtree has.

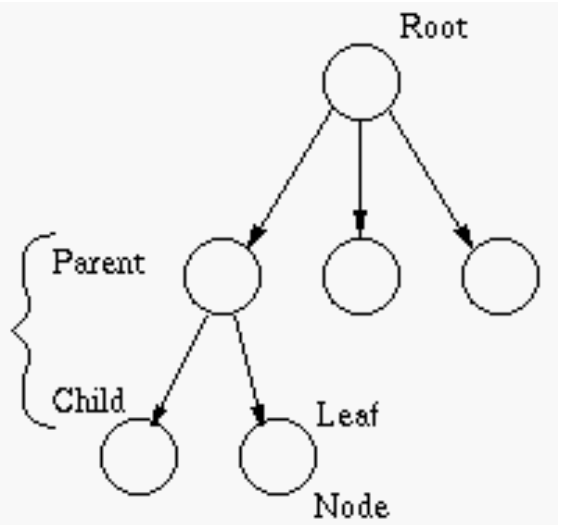
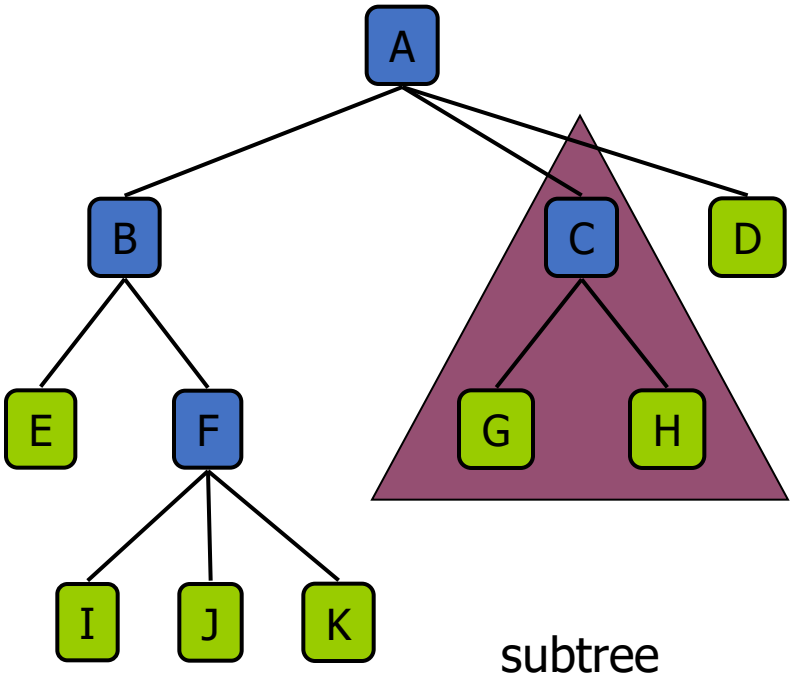


Figure: Tree data structure

Tree Terminology

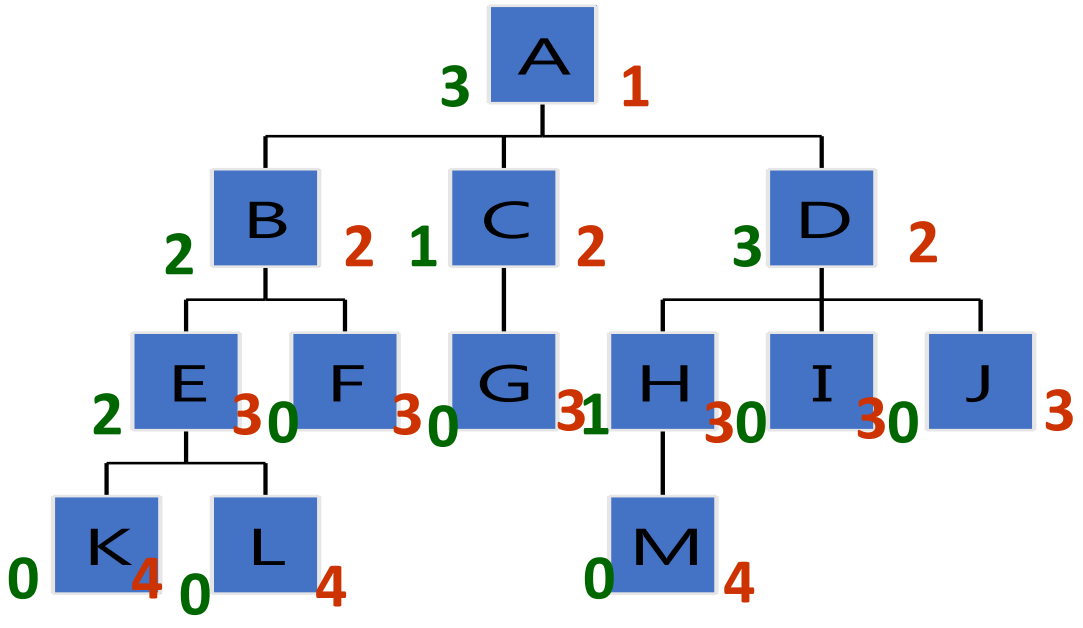
- **Root**: node without parent (A)
- **Siblings**: nodes share the same parent
- **Internal node**: node with at least one child (A, B, C, F)
- **External node** (leaf): node without children (E, I, J, K, G, H, D)
- **Ancestors** of a node: parent, grandparent, grand-grandparent, etc.
- **Descendant** of a node: child, grandchild, grand-grandchild, etc.
- **Depth** of a node: number of ancestors
- **Height** of a tree: maximum depth of any node (3)
- **Degree** of a node: the number of its children
- **Degree** of a tree: the maximum degree of any of its nodes.

- **Subtree**: tree consisting of a node and its descendants



Level and Depth

node (13)
degree of a node
leaf (terminal)
nonterminal
parent
children
sibling
degree of a tree (3)
ancestor
level of a node
height of a tree (3)

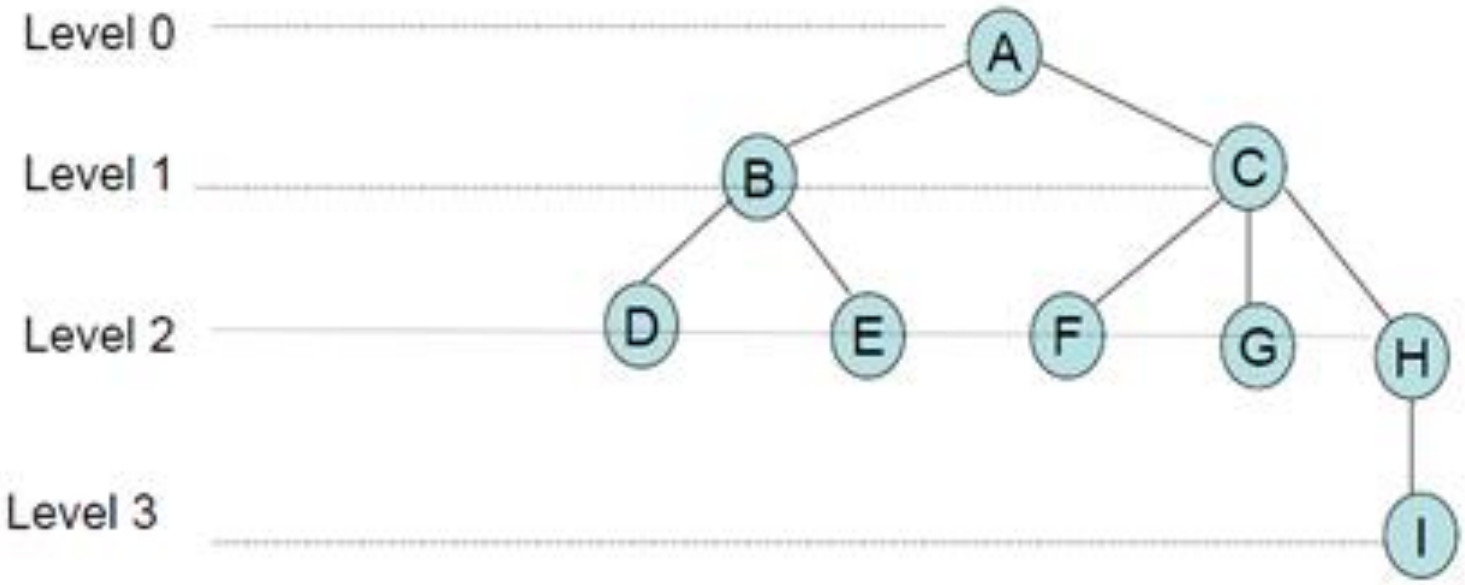


Terminology

- The degree of a node is the number of subtrees of the node
  - The degree of A is 3; the degree of C is 1.
- The node with degree 0 is a leaf or terminal node.
- A node that has subtrees is the *parent* of the roots of the subtrees.
- The roots of these subtrees are the *children* of the node.
- Children of the same parent are *siblings*.
- The ancestors of a node are all the nodes along the path from the root to the node.

Depth

The **depth** of a node is the length of the path from the root to the node. The set of all nodes at a given depth is sometimes called a **level** of the tree. The root node is at depth zero. Eg. The depth of E is 2.

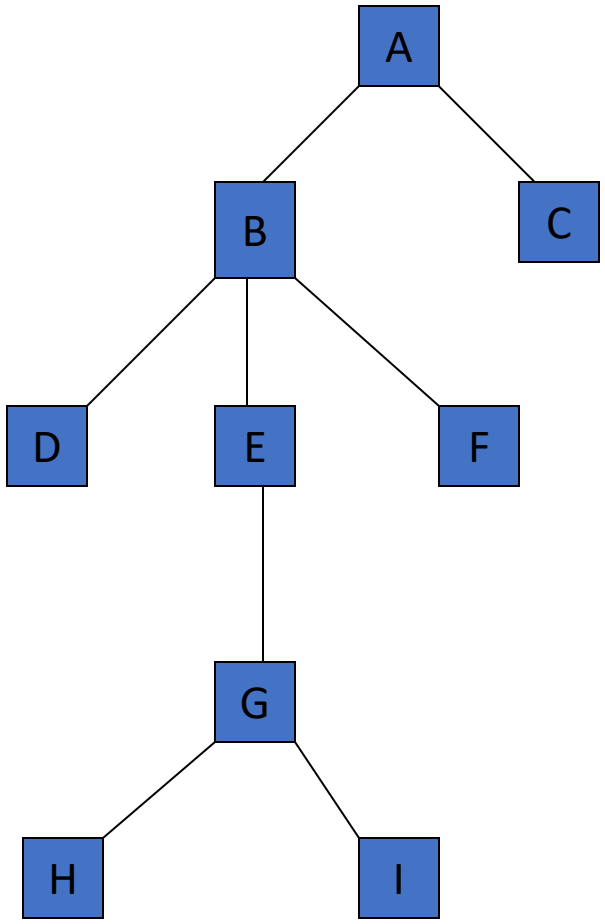


Height

The **height** of a tree is the length of the path from the root to the deepest node in the tree. A (rooted) tree with only a node (the root) has a height of zero.

Eg.The height of the given tree is 3.

Tree Properties



Property	Value
Number of nodes	
Height	
Root Node	
Leaves	
Interior nodes	
Ancestors of H	
Descendants of G	
Siblings of E	
Right subtree of A	
Degree of this tree	

Application Area

- Manipulate hierarchical data (operating systems store files in trees or tree like structures. the directory structure in many common operating systems, including Windows, Unix and DOS).
- As a workflow for compositing digital images for visual effects.
- Router algorithms
- Compiler Design/Text processing
- Searching Algorithms
- Evaluating mathematical expressions
- Analysis of electrical circuits.

Binary Tree

Binary Tree

- A tree whose elements have at most 2 children is called a binary tree. Since each element in a binary tree can have only 2 children, we typically name them the left and right child.
- Binary trees are excellent data structures for searching large amounts of information. They are commonly used in database applications to organize key values that index database records.
- When used to facilitate searches, a binary tree is called a binary search tree.

Properties of binary trees

A binary tree can contain at most 2^L nodes at level L

At level 0 B-tree can contain at most 1= 2^0 nodes
At level 1 B-tree can contain at most 2= 2^1 nodes
At level 2 B-tree can contain at most 4= 2^2 nodes
At level L B-tree can contain at most -> 2^L nodes

If a binary tree contains m nodes at level L, then it contains at most 2^m nodes at level L+1

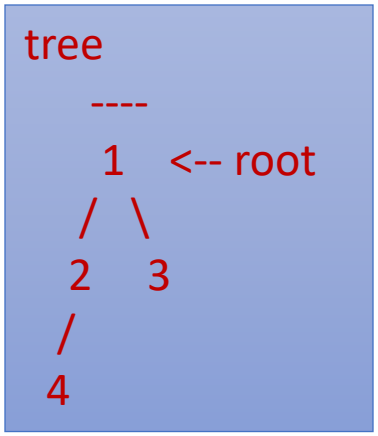
Binary Tree Representation

A tree is represented by a pointer to the topmost node in tree. If the tree is empty, then value of root is NULL. A Tree node contains following parts.

1. Data
2. Pointer to left child
3. Pointer to right child

```
struct node
{
  int data;
  struct node *left;
  struct node *right;
};
```

Tree in C




```
struct node
{
    int data;
    struct node *left;
    struct node *right;
};

/* newNode() allocates a new node with the given data and NULL
left and
   right pointers. */
struct node* newNode(int data)
{
  // Allocate memory for new node
  struct node* node = (struct node*)malloc(sizeof(struct node));

  // Assign data to this node
  node->data = data;

  // Initialize left and right children as NULL
  node->left = NULL;
  node->right = NULL;
  return(node);
}
```

```
int main()
{
  /*create root*/
  struct node *root = newNode(1);

 /* following is the tree after above
statement

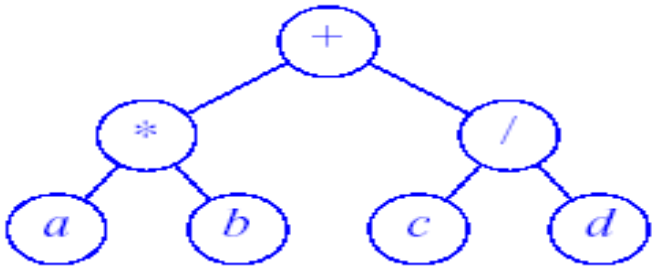
      1
    /   \
   NULL  NULL
 */


 root->left       = newNode(2);
 root->right      = newNode(3);
 /* 2 and 3 become left and right children of
1
        1
      /   \
     2     3
   /   \   / \
  NULL NULL NULL NULL
 */
```

```
 root->left->left  = newNode(4);
 /* 4 becomes left child of 2
        1
      /     \
     2       3
   /   \     / \
  4    NULL  NULL  NULL
 / \
NULL NULL
*/

return 0;
}
```

Binary Tree for Expressions



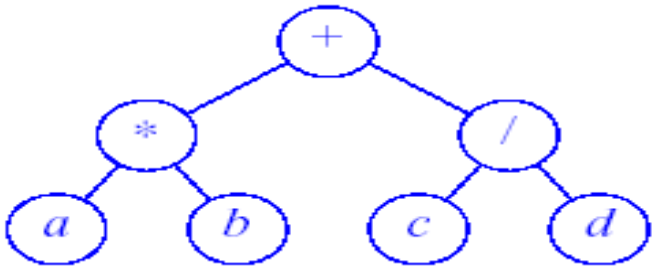
(a) $(a * b) + (c / d)$

(b) $((a + b) + c) + d)$

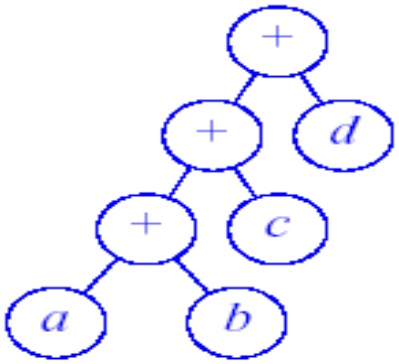
(c) $((-a) + (x + y)) / ((+b) * (c * a))$

Expression Trees

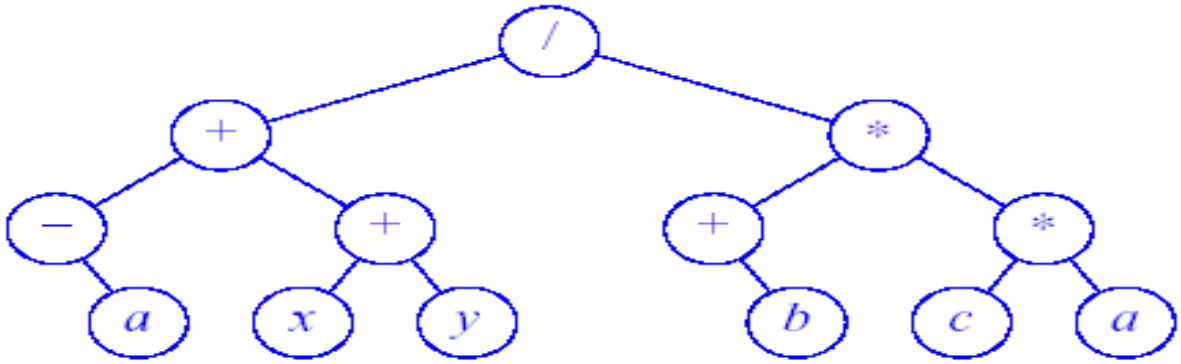
Binary Tree for Expressions



(a) $(a * b) + (c / d)$



(b) $((a + b) + c) + d)$



(c) $((-a) + (x + y)) / ((+b) * (c * a))$

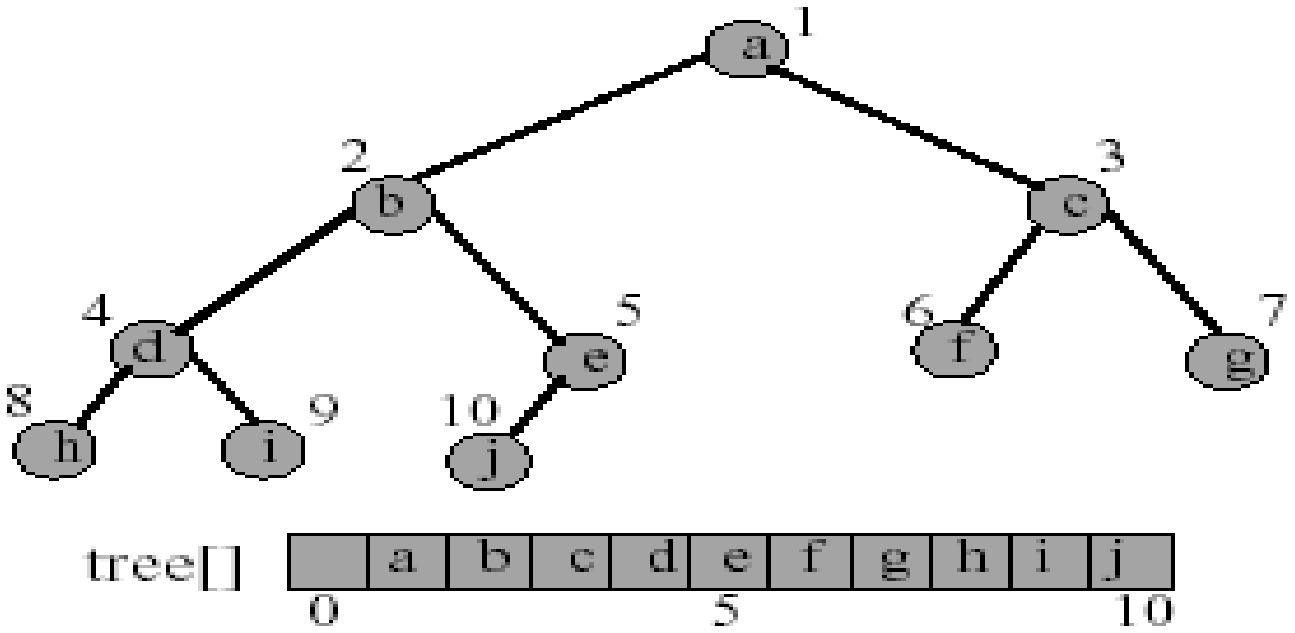
Expression Trees

Binary Tree Representation

- Array representation
- Linked representation

Array Representation of Binary Tree

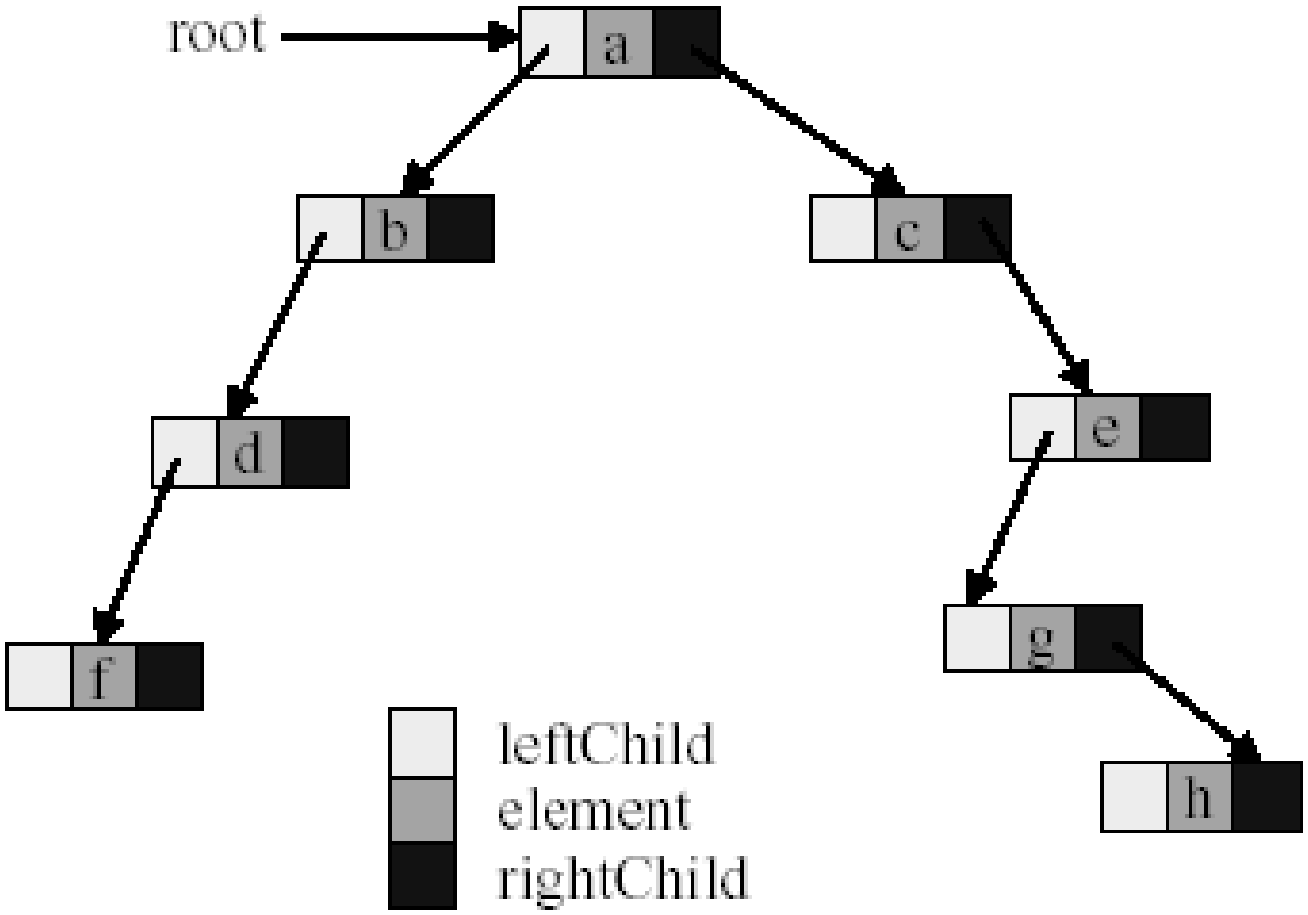
- The binary tree is represented in an array by storing each element at the array position corresponding to the number assigned to it.



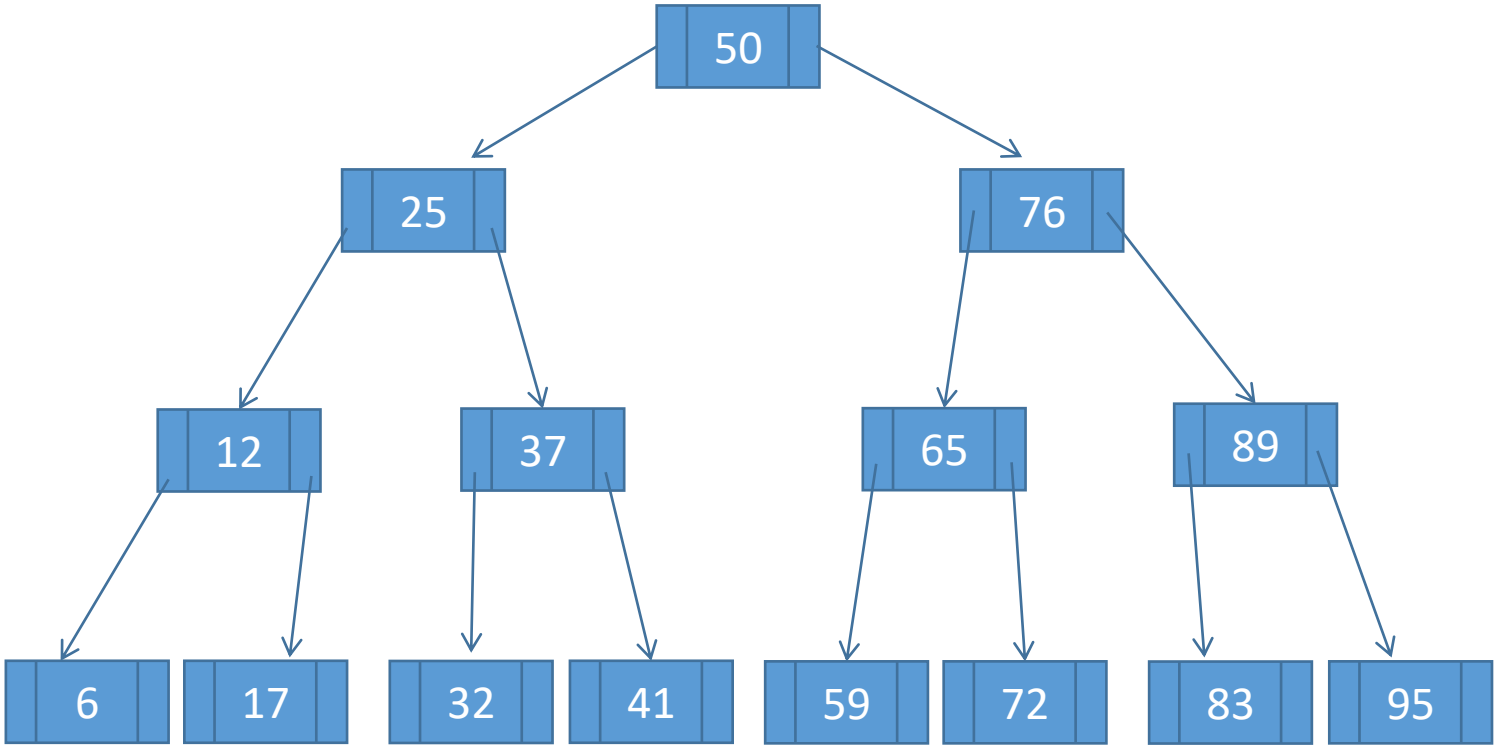
Linked Representation of Binary Tree

- The most popular way to present a binary tree
- Each element is represented by a node that has two link fields (leftChild and rightChild) plus an element field .
- Each binary tree node is represented as an object whose data type is binaryTreeNode .
- The space required by an *n* node binary tree is *n* * sizeof(binaryTreeNode)

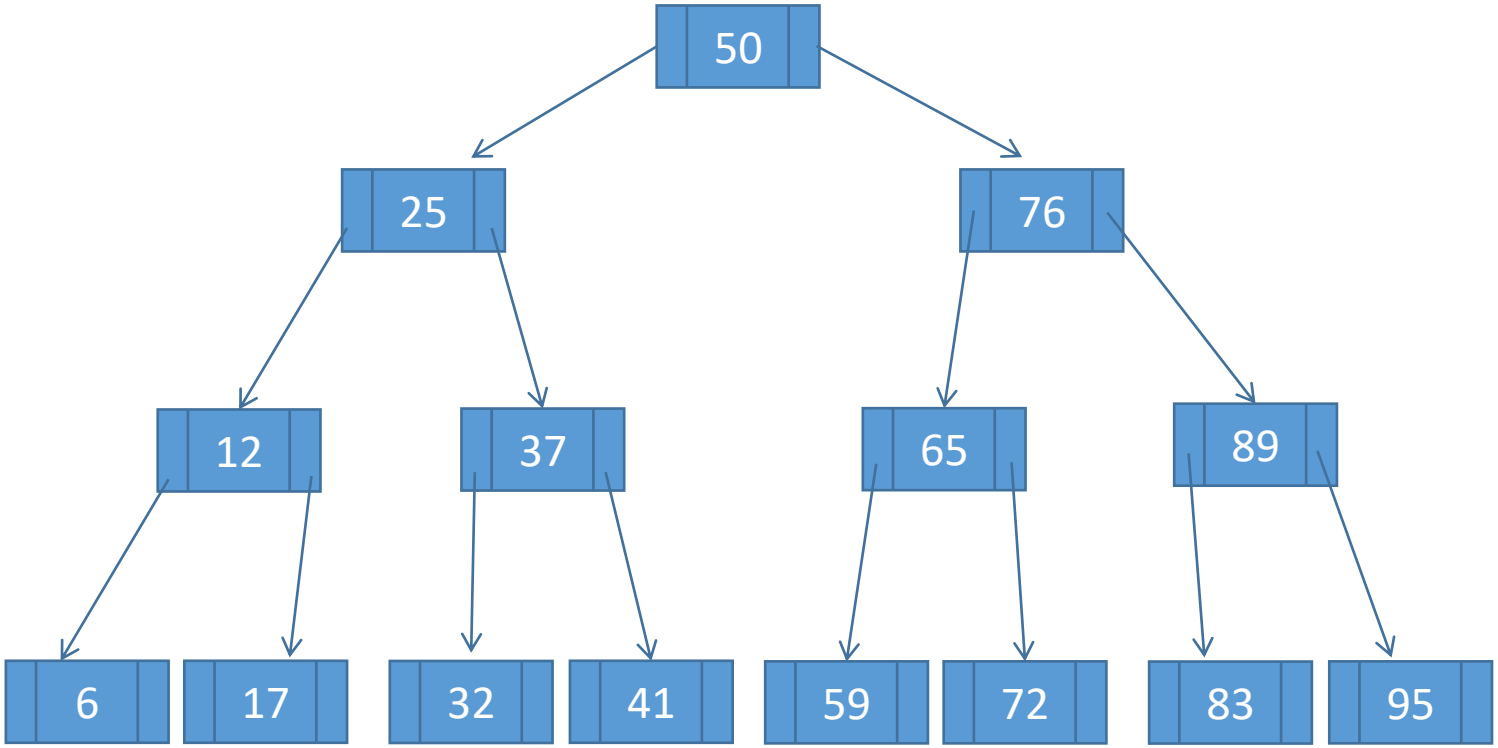
Linked Representation of Binary Tree



Binary search tree



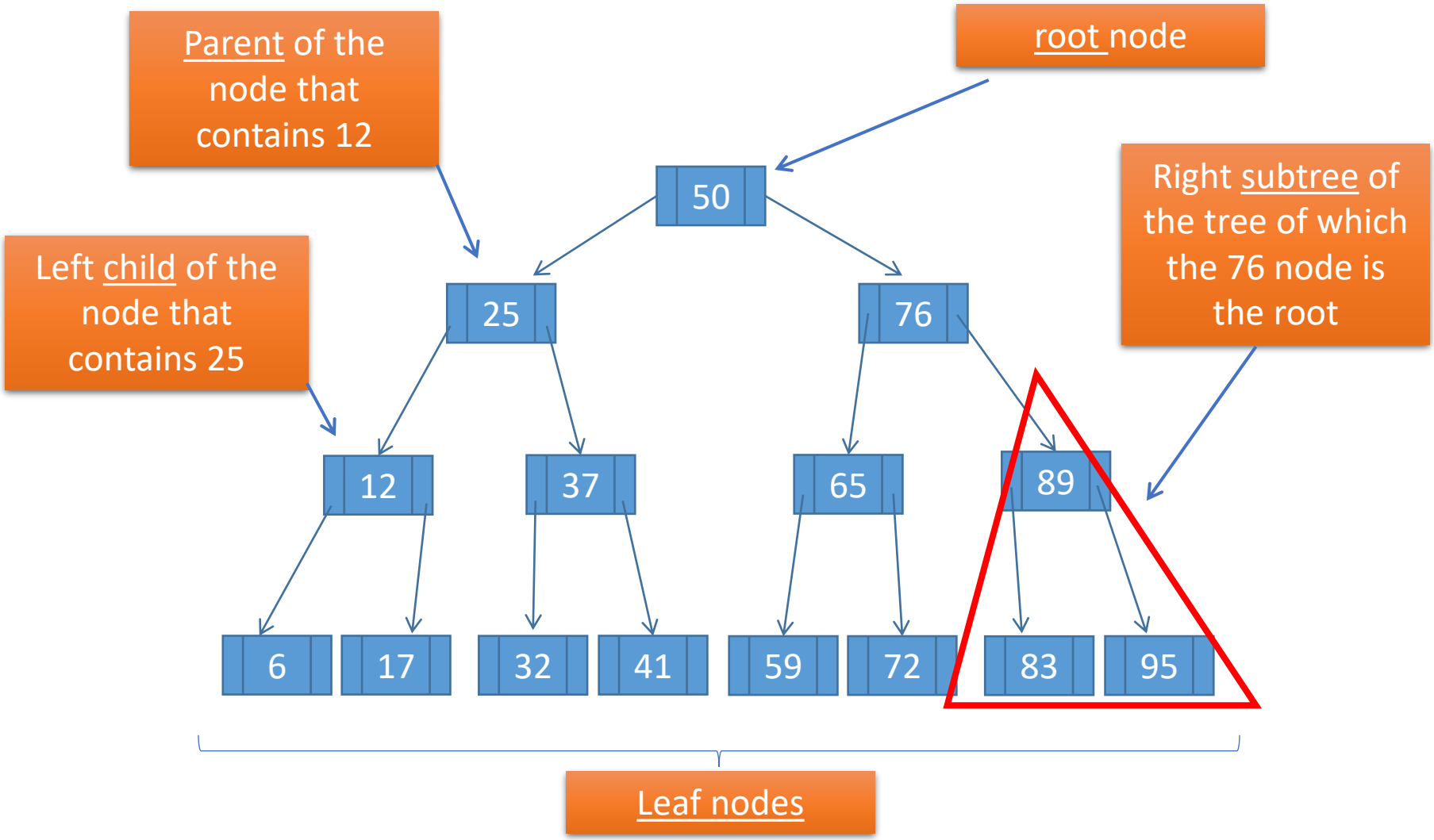
Binary search tree



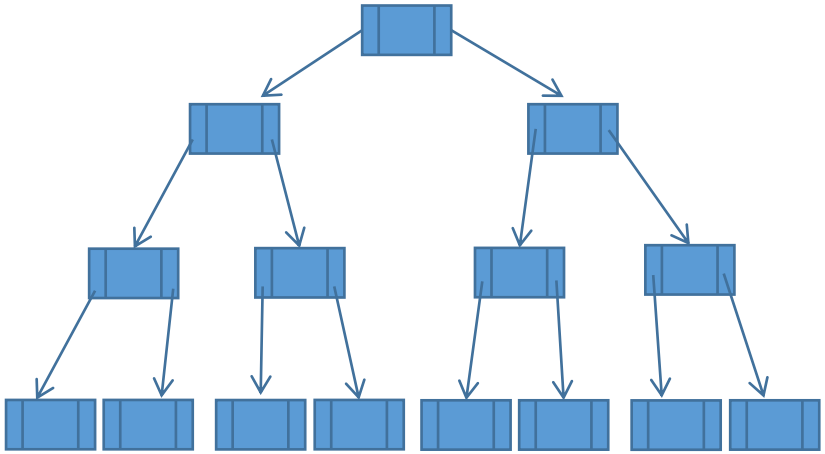
Binary search tree: A binary tree that has the following properties:

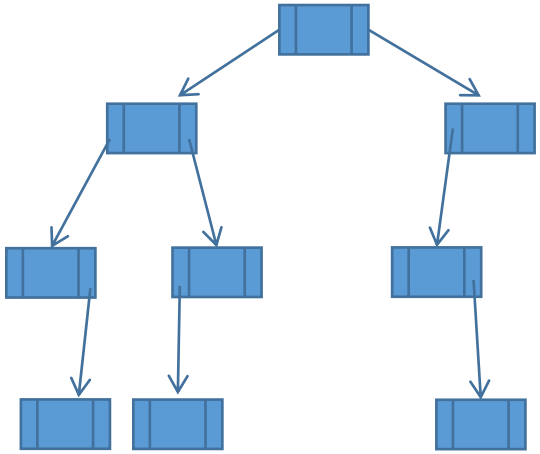
- The left subtree of a node contains only nodes with data less than the node's data.
- The right subtree of a node contains only nodes with data greater than the node's data.
- Both the left and right subtrees are also binary search trees.

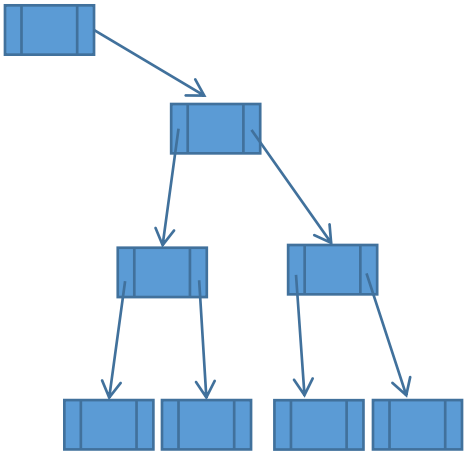
Terminology

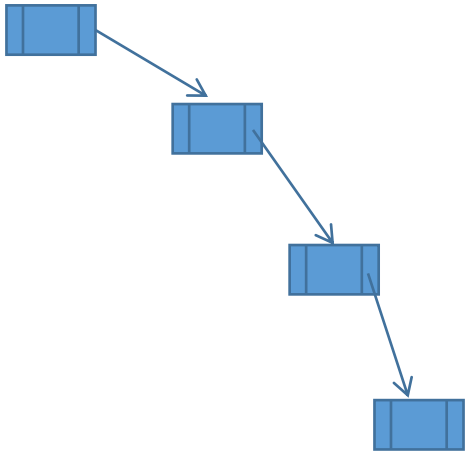


Binary trees don’t have to be symmetrical



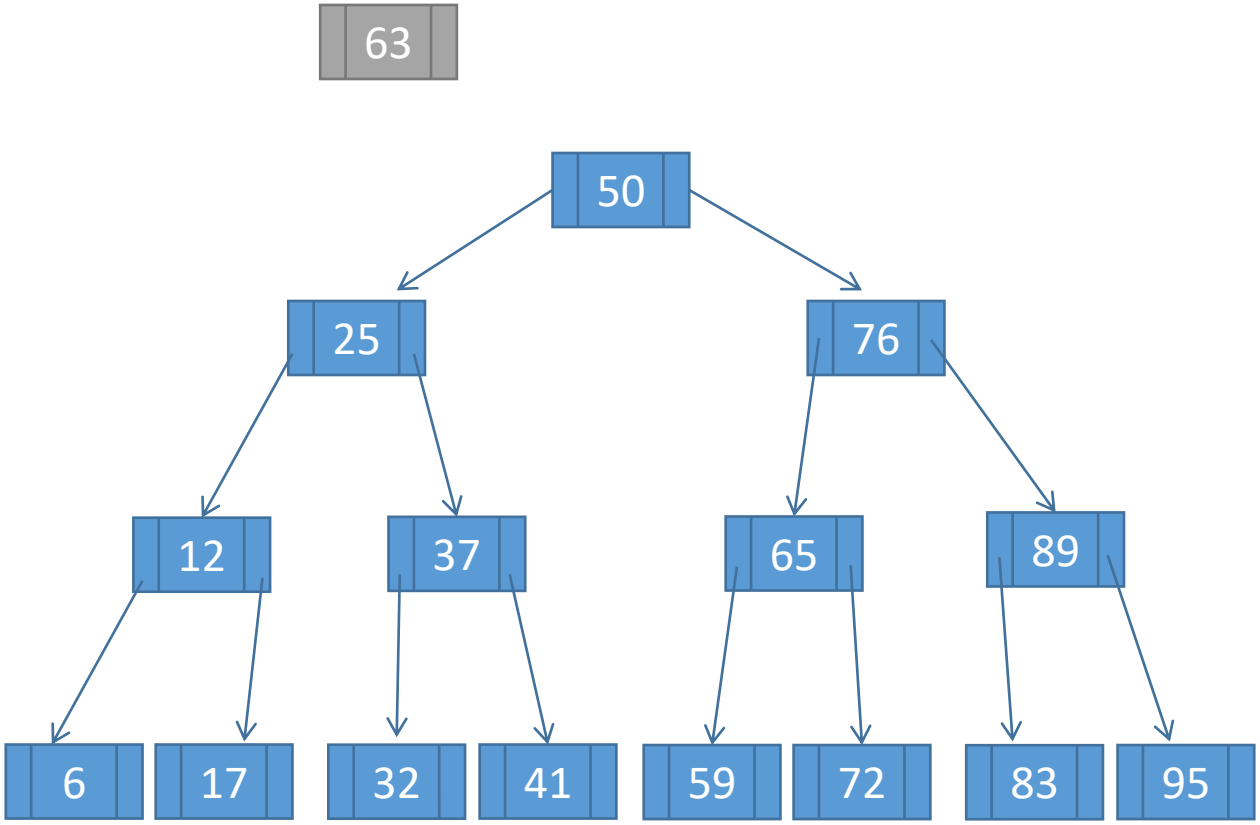






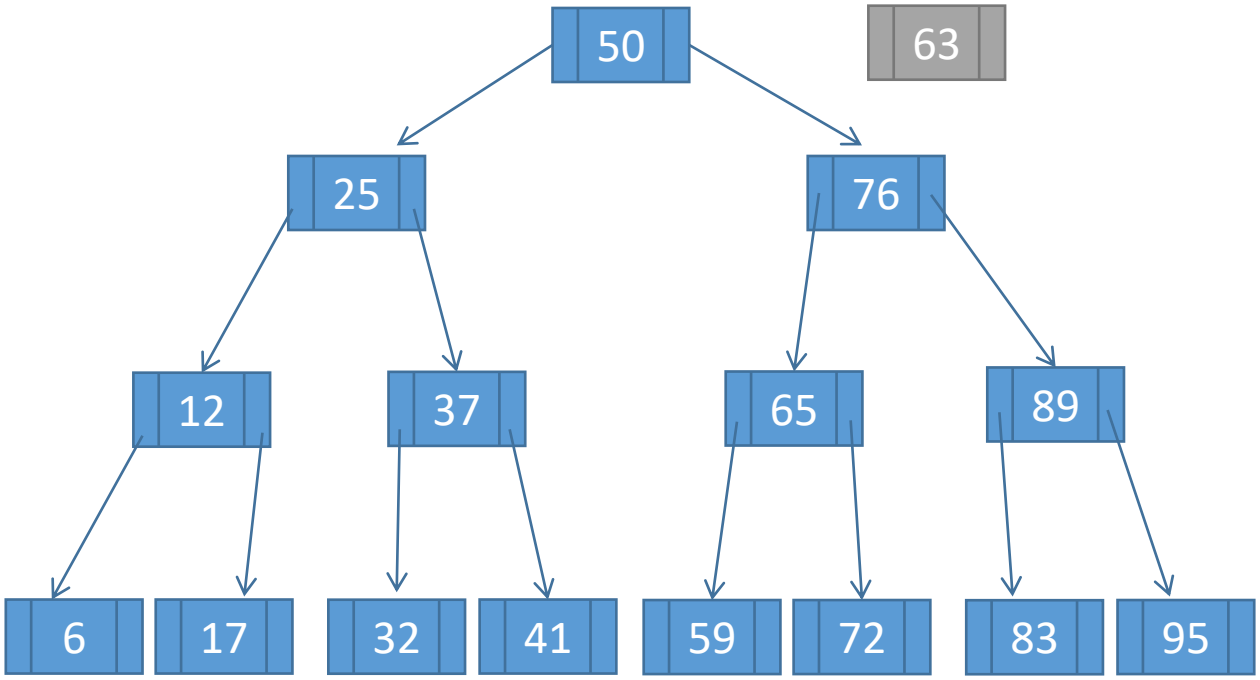
Insertion operation

We want to insert a new node into the tree. Where should we put it?



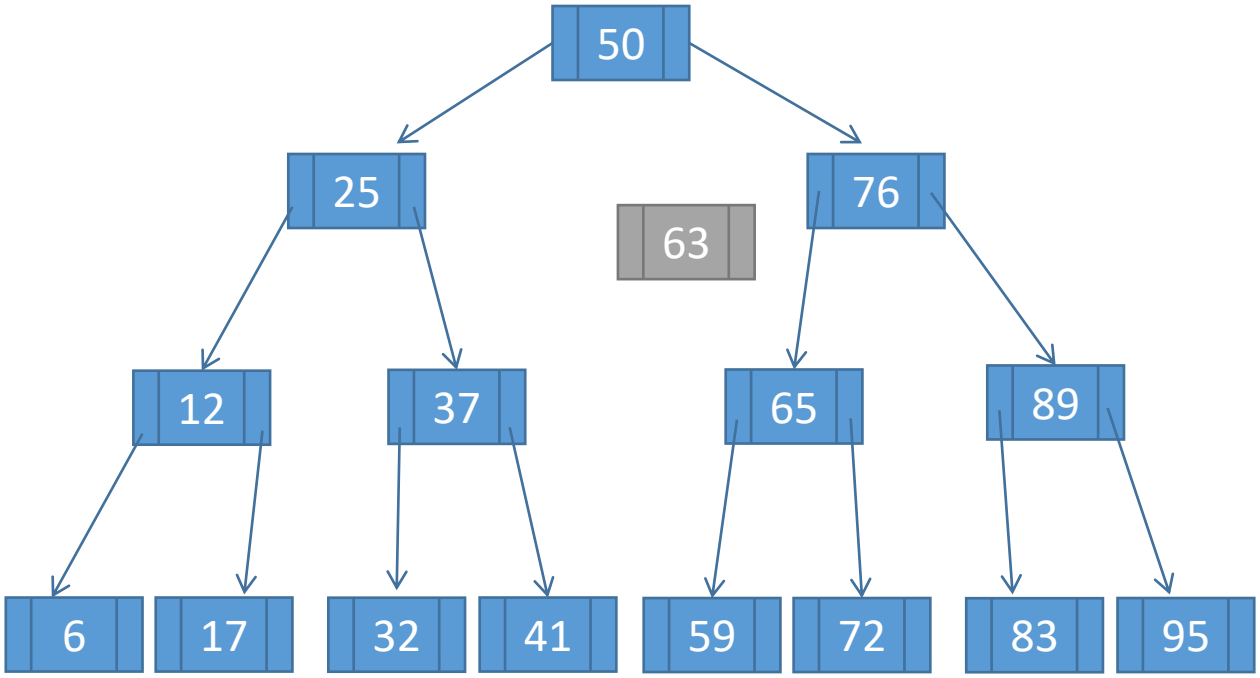
Insertion operation

It is bigger than 50, so go down the right subtree…



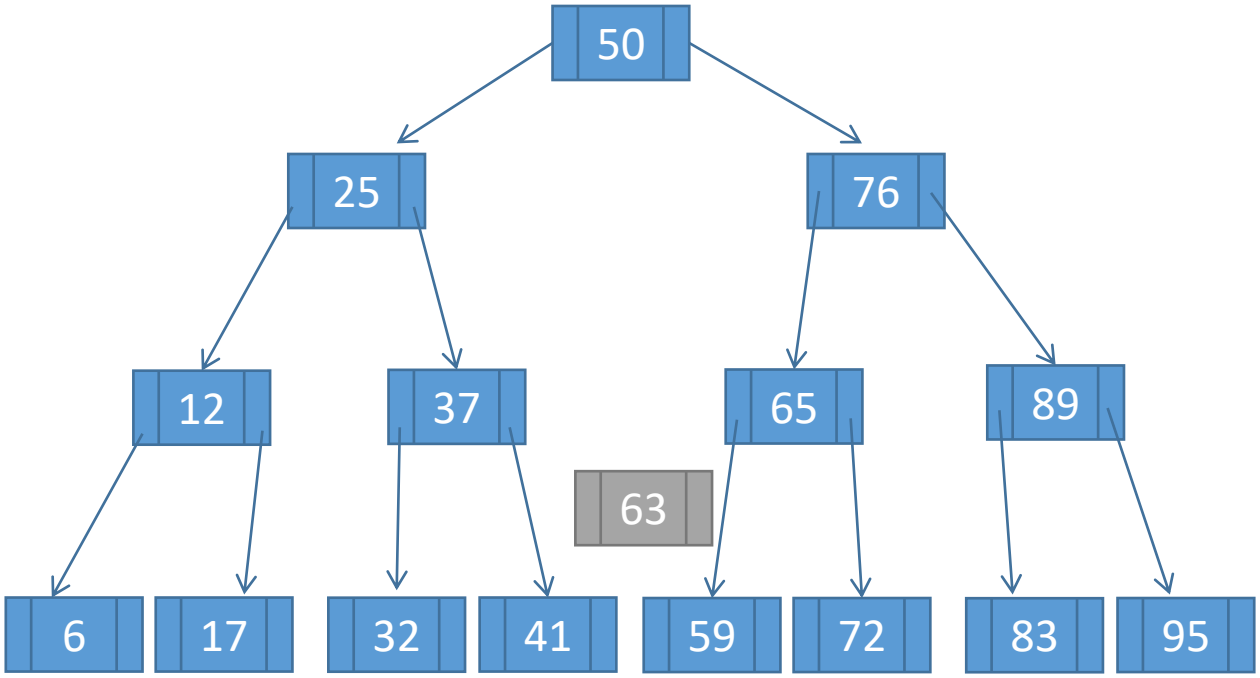
Insertion operation

It is smaller than 76, so go down the left subtree…



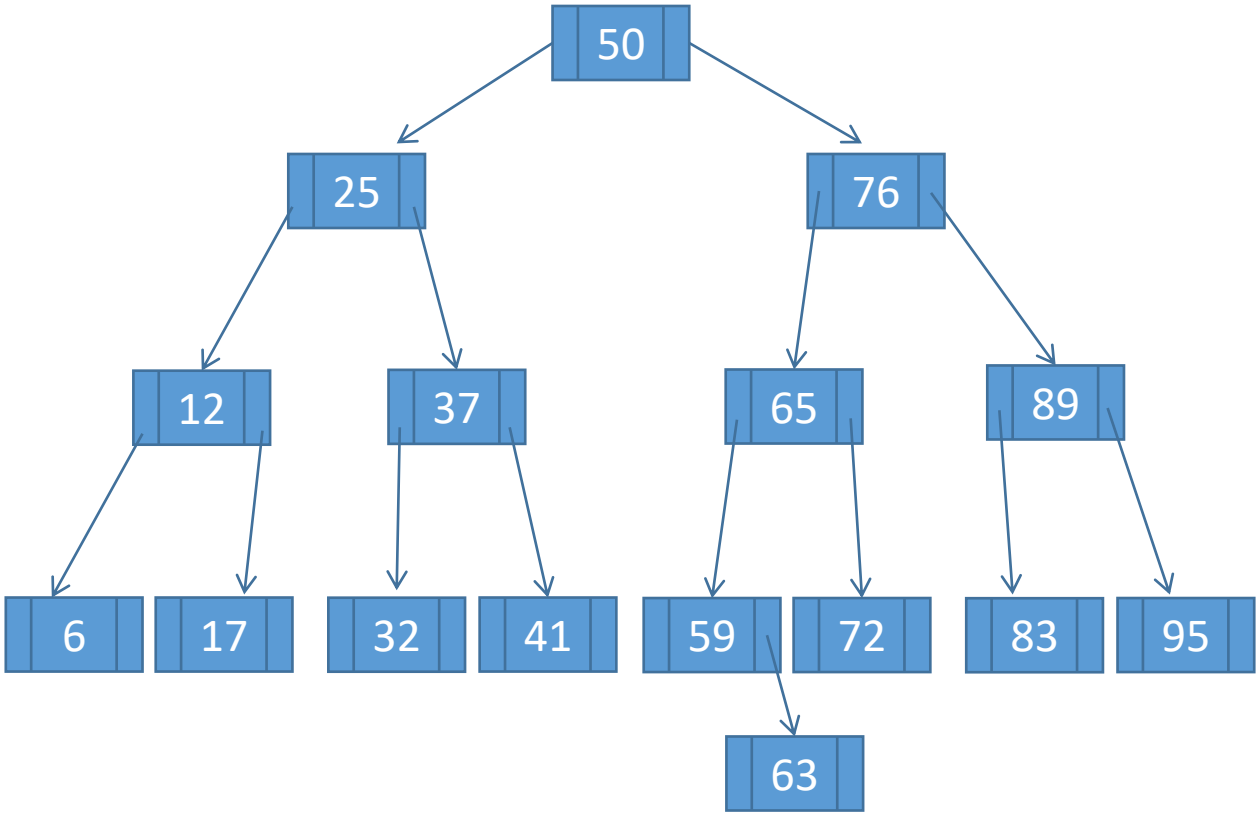
Insertion operation

It is smaller than 65, so go down the left subtree…



Insertion operation

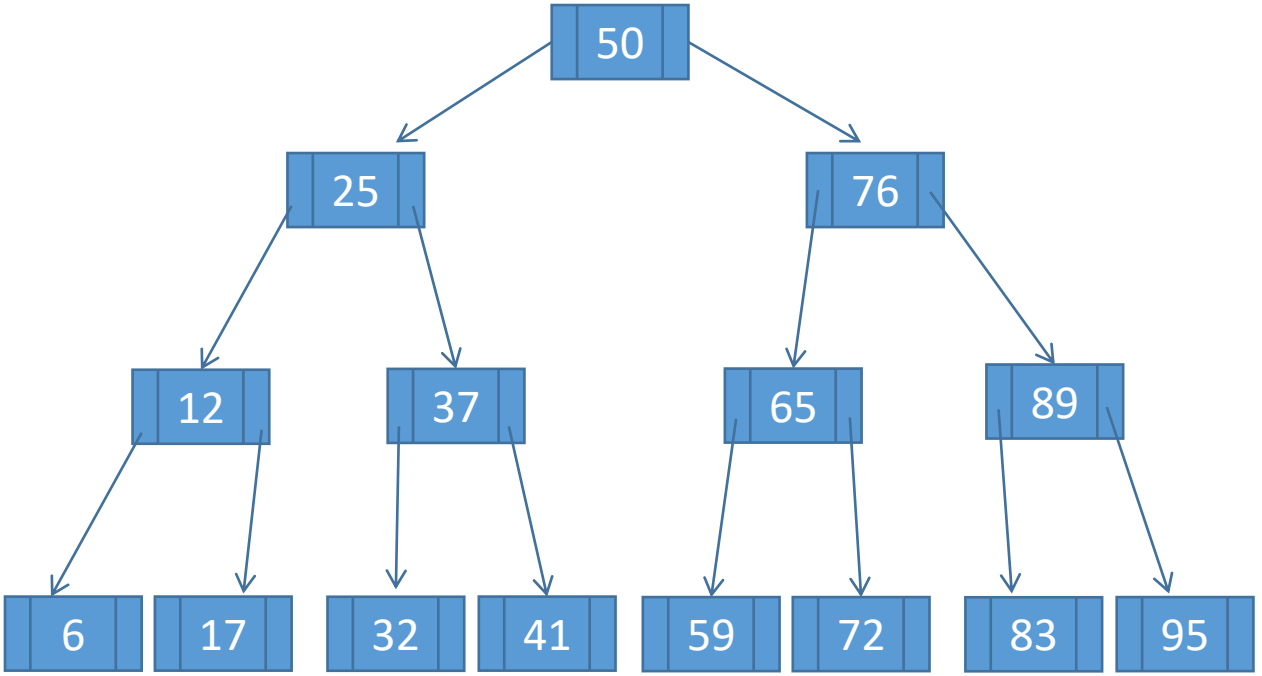
It is bigger than 59, and 59 has no right children, so that's where it goes.



Binary Tree Traversals

There are three types of traversal:

- Preorder
- Inorder
- Postorder



END